Virtualisation & Cloud

Chapitre 4: Kubernetes

Dr Dhekra EL HAMDI

Contact: hamdi.dhikra@yahoo.fr

2025/2026

Plan

1. Introduction
2. Histoire de Kubernetes
3. Orchestration des conteneurs
4. Les objets Kubernetes
5. Les composants des nodes
6. Cluster Kubernetes

Introduction

- Imaginez que vous voulez déployer une application sur plusieurs serveurs. Comment vous assurer que chaque version est bien déployée et à jour sur toutes les machines ?
- Si un serveur tombe en panne, comment vous assurer que votre application continue de fonctionner sans intervention manuelle ?
- Si votre application est constituée de plusieurs services (API, base de données, frontend), comment pouvez-vous gérer efficacement tous ces conteneurs sur plusieurs machines ?
- Que se passe-t-il si votre application soudainement beaucoup reçoit de trafic ?
- pouvez-vous ajouterautomatiquement plus de ressources sans tout redémarrer ?

Pourquoi Kubernetes ?

Face à tous ces défis, une solution existe : Kubernetes (K8s)

Kubernetes (souvent abrégé en K8s) est un système open-source de gestion et d'orchestration de conteneurs qui :

- Automatiser le déploiement, la mise à l'échelle et la gestion d'applications conteneurisées.
- Gère la montée en charge (scalabilité automatique).
- Assure la haute disponibilité et la reprise après panne.
- Facilite la mise à jour continue et le rollback.

Un peu d'histoire...

L'histoire de **Kubernetes** commence chez **Google**.

Google avait besoin d'une infrastructure massive pour rendre accessible au monde entier :

- Son moteur de recherche,
- Ses services comme Gmail, YouTube,
- Et surtout son système de publicités qui génère l'essentiel de ses revenus.

Pour répondre à ces besoins, Google a développé un système interne d'orchestration de conteneurs appelé **Borg**.

Fort de cette expérience, Google a ensuite créé et rendu open source **Kubernetes** en 2014.

Orchestration des conteneurs

Caractéristiques	Kubernetes	Docker Compose
Objectif principal	Orchestration de conteneurs à grande échelle, utilisé pour la gestion de clusters distribués.	Orchestration simple de conteneurs sur une seule machine.
Complexité	Très complexe, adapté aux systèmes distribués à grande échelle.	Simple et facile à utiliser pour les petits environnements locaux.
Déploiement	Multi-nœuds (cluster de machines), gestion automatique du réseau, stockage, etc.	Mono-nœud (en général, pour un usage local ou développement).
Gestion des nœuds	Gère plusieurs nœuds (serveurs) dans un cluster. Peut équilibrer les charges entre différents nœuds.	Fonctionne généralement sur une seule machine.

Réseau	Réseau inter-nœuds Automatique et configurable pour les pods dans tout le cluster.	Réseau simple entre conteneurs sur la même machine via des *network* Docker.
Tolérance aux pannes	Tolérance aux pannes intégrée, recrée les pods sur d'autres nœuds si nécessaire.	Pas de mécanisme intégré pour la tolérance aux pannes. restart: always
Configuration	Configuration complexe via des fichiers YAML avec plusieurs objets (pods, services, etc.).	Fichiers YAML simples (docker-compose.yml) pour définir les services, réseaux et volumes.

Les objets Kubernetes: Simplifier la gestion des conteneurs (1/2)

Kubernetes a pour objectif principal de **dissimuler la complexité** de la gestion d'un parc de conteneurs.
Pour cela, plusieurs objets essentiels existent pour vous faciliter la vie :

- **Node** : Une machine de travail du cluster (physique, virtuelle ou cloud).
- **Pod** : L'unité de base de Kubernetes. Chaque pod contient un ou plusieurs conteneurs partageant la même stack réseau et le même stockage.
- **Replicas** : Nombre d'instances d'un Pod (réplications de vos applications).
- **ReplicaSet** : Garantit que le nombre souhaité de réplicas d'un pod est en fonctionnement.

Les objets Kubernetes: Simplifier la gestion des conteneurs (2/2)

Suite des objets clés :

- **Deployment** : Décrit l'état souhaité de l'application et permet les mises à jour automatiques des Pods et ReplicaSets.
- **Service** : Regroupe logiquement un ensemble de pods.
  - Donne une IP fixe et un nom DNS unique.
  - Permet la configuration du **Load Balancing** pour mieux gérer la montée en charge.

Fichier yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

Architecture de Kubernetes

Kubernetes suit une architecture **maître-esclave** :

- Le **Master** (ou Control Plane) gère l'ensemble du cluster. Il prend les décisions globales (planification, déploiement, surveillance).
- Les **Workers** (ou *Minions*) exécutent les applications. Leur rôle principal est de fournir la capacité de calcul. Ils ne prennent pas de décisions et n'ont aucun pouvoir sur les autres nœuds.

Les composants clés du Master et des Workers seront détaillés dans la prochaine section.

Les composants du Master

Sur un nœud de type **Master**, on retrouve plusieurs composants essentiels :

- **kube-apiserver** :
  Point d'entrée du cluster.
  Permet la communication entre les différents outils et Kubernetes.
- **kube-scheduler** :
  Planifie la répartition des pods sur les nœuds du cluster.
  Prend en compte les ressources disponibles et les besoins des pods.
- **kube-controller-manager** :
  Gère plusieurs controleurs (replication controller, endpoints controller, namespace controller. . .).
  Veille à ce que l'état du cluster corresponde à l'état désiré.
- **cloud-controller-manager** :
  Gère l'intégration avec les fournisseurs de cloud (AWS, Azure, GCP...).
- **etcd** :
  Base de données clé-valeur distribuée.
  Stocke toute la configuration et l'état du cluster.

Les composants du Worker

Sur chaque **Worker Node**, on retrouve les composants suivants :

- **kubelet** :

  Agent principal sur chaque nœud.
  Communique avec le Master et consulte etcd pour connaitre les taches à exécuter.
  Veille au bon fonctionnement des conteneurs d'un Pod.
  Redémarre les conteneurs en cas de défaillance.

- **kube-proxy** :

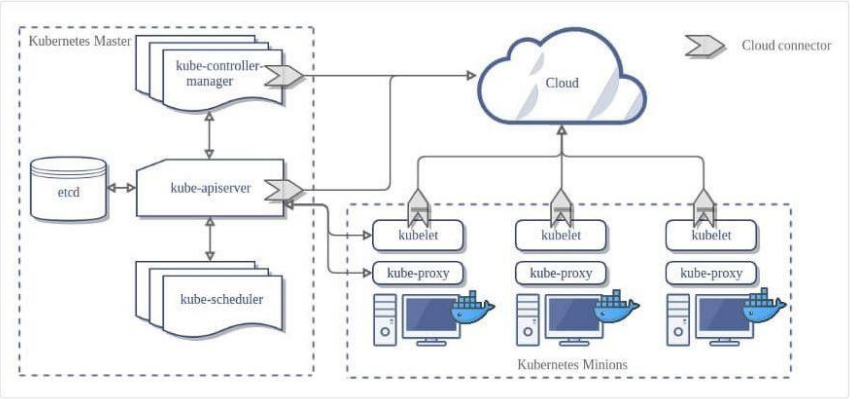
  Gère les règles réseau sur chaque nœud.
  Permet l'abstraction réseau des Services Kubernetes.
  Facilite l'accès aux Pods depuis l'extérieur du cluster.

- **Environnement d'exécution de conteneurs** :

  Exécute les conteneurs (par défaut Docker, mais Kubernetes supporte aussi d'autres moteurs comme *containerd* ou *rkt*).

Architecture du Cluster Kubernetes

Si l'on reprend tous les composants vus précédemment, l'architecture du cluster Kubernetes ressemble à ceci :



Cluster Kubernetes

Un **cluster Kubernetes** est l'ensemble de toutes les ressources physiques ou virtuelles (nœuds) qui travaillent ensemble pour exécuter et gérer des applications conteneurisées. Il comprend à la fois des **Master Nodes** (pour gérer et orchestrer le cluster) et des **Worker Nodes** (qui exécutent réellement les applications et conteneurs).

Composants du cluster :

- **Master Node (Control Plane)** : Gère la logique de contrˆole du cluster.
- **Worker Nodes** : Hébergent les conteneurs, exécutent des Pods, etc.

En d'autres termes, le **cluster Kubernetes** représente l'infrastructure sous-jacente et les ressources qui hébergent et gèrent les applications conteneurisées dans Kubernetes.

Communication avec le cluster

Pour communiquer avec le cluster Kubernetes, on peut utiliser le binaire kubectl, ou une interface web de management nommée Dashboard.

kubectl (ligne de commande) :

L'outil en ligne de commande Kubernetes, kubectl, vous permet d'exécuter des commandes sur les clusters Kubernetes afin de créer et gérer des objets k8s et interagir avec l'API Kubernetes.

Vous pouvez utiliser kubectl pour déployer des applications, inspecter et gérer les ressources du cluster, et afficher les logs.

Dashboard (interface web) :

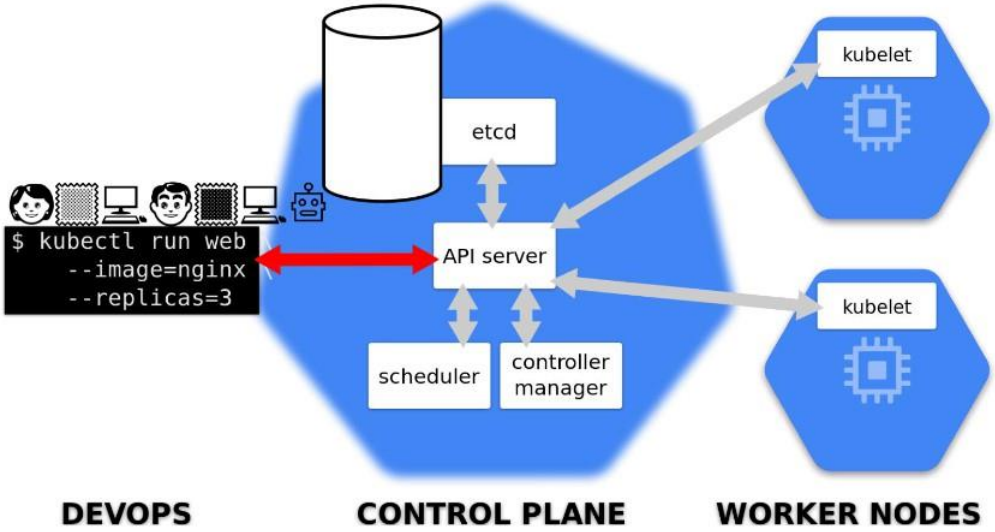
Dashboard est l'interface utilisateur Web officielle de Kubernetes.

Il fournit une vue d'ensemble des applications en cours d'exécution sur votre cluster Kubernetes.

Il permet aux administrateurs de surveiller et gérer les opérations de leurs clusters Kubernetes, dépanner les applications conteneurisées et gérer les ressources du cluster.

Requête pour exécuter un conteneur dans un cluster Kubernetes

- L'administrateur envoie une requête via kubectl pour lancer un conteneur (en créant un pod ou un déploiement).
- L'API Server valide et enregistre la demande.
- Le Scheduler sélectionne le nœud pour exécuter le conteneur.
- Le Kubelet sur ce nœud exécute le conteneur à l'aide d'un runtime de conteneur.


etcd
kubelet
$ kubectl run web
--image=nginx
--replicas=3
API server
kubelet
scheduler
controller
manager
DEVOPS
CONTROL PLANE
WORKER NODES

Exemple : Recréation d'un pod

- Détection de l'échec :

Si un pod échoue ou si un nœud devient indisponible, le Controller Manager (via le ReplicaSet ou Replication Controller) détecte qu'il manque un pod par rapport à l'état souhaité.

- Décision de recréation :

Le Replication Controller ou ReplicaSet signale qu'il manque un pod et demande à Kubernetes d'en recréer un.

- Planification du nouveau pod :

Le Scheduler recoit l'information qu'un nouveau pod doit être créé. Il évalue les ressources disponibles dans le cluster et choisit un nœud approprié ou déployer le pod.

- Exécution du pod :

Une fois le nœud choisi, le Kubelet sur ce nœud prend en charge l'exécution du pod, en téléchargeant les conteneurs et en les exécutant selon la configuration définie.

Namespace

- Les namespaces sont des mécanismes pour diviser un cluster en environnements logiques distincts.
- Ils permettent de gérer les ressources de manière isolée, utile pour séparer les environnements de développement, de test et de production au sein d'un même cluster.

ConfigMap et et Secret

- ConfigMap : Permet de stocker des données de configuration sous forme de paires clé-valeur. Ces données peuvent être injectées dans les conteneurs ou les pods en tant que variables d'environnement ou fichiers.
- Secret : Comme un ConfigMap, mais il est utilisé pour stocker des données sensibles (par exemple des mots de passe, des clés API), et ces données sont encodées.

Volume

- PersistentVolume (PV) permet de persister les données au-del à de la durée de vie des conteneurs.
- Par défaut, les conteneurs sont éphémères et les données stockées à l'intérieur sont perdues lorsqu'ils sont arrêtés. Un volume permet de lier des données persistantes à un pod, garantissant leur durabilité.